

Regular Expressions Basics

The re module — pattern matching, searching, and text extraction

■ What is Regex?

Regular Expressions (often abbreviated as **Regex** or **RegEx**) are powerful tools for searching, pattern matching, validating, and extracting specific text within strings.

Imagine searching for a **pattern** instead of an exact string:

- **Standard search:** finds exact matches like "cat" in "The cat sat".
- **Regex search:** can find patterns like "any 10-digit phone number", "a valid email address", or "all words starting with 'Py'".

Python provides the built-in **re** module to work with regular expressions.

■■ Essential Regex Patterns & Symbols

Here are the primary building blocks for creating regex patterns.

1. Special Sequences (Character Classes)

Symbol	Matches	Example Match
\d	Any single digit (0–9)	"5" in "Code5"
\D	Any non-digit	"A" in "5A"
\w	Any letter, digit, or underscore (a-z, A-Z, 0-9, _)	"py_3"
\W	Any non-word character (spaces, punctuation)	"!", " "
\s	Any whitespace (spaces, tabs \t, newlines \n)	" "

2. Quantifiers (How Many Times?)

Symbo l	Meaning	Example Pattern	Matches
+	1 or more times	\d+	"123"
*	0 or more times	\d*	"" or "123"

Day 71 — Regular Expressions Basics (re module)

Symbol	Meaning	Example Pattern	Matches
?	0 or 1 time (optional)	colou?r	"color" or "colour"
{n}	Exactly n times	\d{3}	"555"

■ Pro Tip

Always prefix regex pattern strings with an **r** (raw string) in Python — e.g. `r"\d+"` — so backslashes `\` aren't accidentally processed as standard Python string escapes!

■ Core re Module Functions

1. `re.search(pattern, text)`

Scans through a string and returns a Match object for the first match found (or None if no match exists).

```
import re

text = "My order ID is 48291."
match = re.search(r"\d+", text)
if match:
    print("Found match:", match.group())
```

```
Found match: 48291
```

2. `re.findall(pattern, text)`

Returns a list of strings containing all non-overlapping matches in the string.

```
import re

text = "Items cost $45, $120, and $5."
prices = re.findall(r"\d+", text)
print(prices)
```

```
['45', '120', '5']
```

Day 71 — Regular Expressions Basics (re module)

3. re.sub(pattern, replacement, text)

Replaces all occurrences of the pattern with a replacement string.

```
import re

text = "Call me at 555-0199 or 555-0188."
censored = re.sub(r"\d", "*", text)
print(censored)
```

```
Call me at ***-**** or ***-****.
```

■ Interactive Practice Exercise (Day 71)

Create a file named **day71.py** in your project folder and add the following code:

```
import re

text = "Contact support@example.com or admin@domain.org for help."

# 1. Find all digits in this sentence
numbers_text = "There are 12 apples, 5 bananas, and 100 oranges."
digits = re.findall(r"\d+", numbers_text)
print("Extracted Numbers:", digits)

# 2. Extract email addresses using regex
email_pattern = r"[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}"
emails = re.findall(email_pattern, text)
print("Extracted Emails:", emails)

# 3. Mask digits in text
sensitive_data = "Account code is 9876"
masked_data = re.sub(r"\d", "X", sensitive_data)
print("Masked Data:", masked_data)
```

■ Run the script:

```
python day71.py
```

Try running this code in VS Code, and let me know how it goes!

Day 71 — Regular Expressions Basics (re module)

■ Your Coding Challenge: The Username & Code Extractor

Got it — no more multiple-choice questions. You're getting a real coding challenge to test your brain! Read the task, write the code yourself in VS Code, run it, and show your code + terminal output.

Scenario

Imagine you are given this mess of raw text from a server:

```
raw_text = "Users user_101 and user_405 logged in. Secret code: 9988."
```

Your Task

- Write a Python script using the `re` module.
- Extract all digits (numbers) from the text as a list using `re.findall()`.
- Mask (replace) every single digit in `raw_text` with an asterisk `*` using `re.sub()`.
- Print both the extracted numbers and the masked text.

■ Expected Output Format

When your program runs, it should look something like this:

```
Extracted Numbers: ['101', '405', '9988']  
Masked Text: Users user_*** and user_*** logged in. Secret code: ****.
```

■ Take your time

Type out the Python script, run it in your terminal, and send over your solution. Let's see how you do, developer!